Course Project Report: Truss-Tactician-AI

Adversarial Multi-Agent Structural Inspection

Team Details

Sl. No.	Name of Student	Roll Number	Department
1	Chaitanya Khangai	23ME3FP33	Mechanical Engineering
2	Mukul Anissh G	23ME10103	Mechanical Engineering
3	Shubham Kumar	23ME30059	Mechanical Engineering
4	Tanmay Gaurav	23ME3FP28	Mechanical Engineering

1 Problem Formulation

The project addresses the challenge of **Autonomous Structural Health Monitoring (SHM)**. In the mechanical domain, inspecting large-scale bridge trusses is hazardous and prone to human error. We model this as a multi-agent system where two drones must navigate complex truss geometries (Pratt or Warren) to scan high-priority damage points. The environment is characterized as **deterministic, static, discrete, and constrained**.

- **Inputs:** Truss coordinates, obstacle maps, and crack severity weights.
- **Outputs:** Optimal, collision-free flight trajectories for dual agents.
- **States:** Discrete 2D grid (x, y) coordinates.

2 Method and Justification

2.1 Why This Method is Appropriate

The choice of **Minimax with Alpha-Beta Pruning** matches the adversarial nature of competition between two drones for limited targets. **A^* Search** is the optimal choice for pathfinding in a discrete grid, as its admissible Manhattan heuristic ensures the shortest battery-efficient path. Finally, **Constraint Satisfaction (CSP)** ensures real-time safety.

2.2 Algorithm Design and Methodology: Full Pipeline

The information flow in *Truss-Tactician-AI* follows a top-down hierarchy: Strategic Layer, Tactical Layer, and Safety Layer. The overall process integrates target selection with path execution.

Algorithm 1 Overall Inspection Pipeline

Require: Structural Grid G, Defect Set D, Start States S_A, S_B
Ensure: Optimized multi-agent inspection plan

```
Initialize environment and load truss geometry
while Defects D are remaining do
    T ← Select next target via Adversarial Selection
    Path ← Generate trajectory via State-Space Search
    Move ← Validate next step via CSP Safety Check
    Update drone positions and mark defect as scanned
end while
return Final trajectories and performance metrics
```

2.2.1 Adversarial Search (Target Selection)

This layer decides which crack is most valuable by evaluating potential utility scores while predicting the opponent's moves to avoid redundant coverage.

Algorithm 2 Minimax with Alpha-Beta Pruning

```
Require: State s, Depth d, α, β, MaximizingPlayer
if d = 0 or Terminal(s) then return Evaluate(s)
end if
if MaximizingPlayer then
    maxEval ← −∞
    for each target t do
        eval ← Minimax(t, d − 1, α, β, False)
        maxEval ← Max(maxEval, eval)
        α ← Max(α, eval)
        if β ≤ α then break
        end if
    end for
    return maxEval
else
    minEval ← +∞
    for each target t do
        eval ← Minimax(t, d − 1, α, β, True)
        minEval ← Min(minEval, eval)
        β ← Min(β, eval)
        if β ≤ α then break
        end if
    end for
    return minEval
end if
```

2.2.2 State-Space Search (Path Planning)

The tactical layer uses A^* to translate the target coordinate into the shortest collision-free path around physical truss obstacles using the Manhattan distance.

Algorithm 3 A^* Search Algorithm

```
Require: Start S, Goal G, Obstacles O
Initialize OpenList with S, ClosedList empty
while OpenList is not empty do
    n ← node in OpenList with lowest f(n) = g(n) + h(n)
    if n = G then return Path to n
    end if
    Expand n to valid neighbors m ∉ O
    for each neighbor m do
        if m has better g(m) then update f(m) and parent
        end if
    end for
end while
```

2.2.3 Constraint Satisfaction (Real-Time Safety)

The safety layer performs a "Forward Check" before movement to ensure the drone maintains a minimum safety radius from the opponent.

Algorithm 4 CSP Safety Validation

Require: ProposedMove M, OpponentPos P, SafetyRadius R

$Dist \leftarrow |M_x - P_x| + |M_y - P_y|$
if $Dist < R$ **then**
    return *Invalid (Wait State)*
else
    return *Valid (Execute Move)*
end if

3 Implementation Details

The software is modularized into `adversarial.py` (Minimax logic), `search.py` (A^* navigation), and `environment.py` (Truss geometries).
GitHub Repository: `https://github.com/BhigaBilla/Truss-Tactician-AI`

4 Results and Performance Analysis

Experimental Setup: Executed on a system with 16GB RAM and an i7 Processor. We evaluated three scenarios: Case 1 (10x10 Pratt), Case 2 (20x20 Pratt), and Case 3 (30x30 Warren). Results demonstrate that A^* ensures optimality in path length while Alpha-Beta pruning effectively reduces computation time as defect density increases.

Table 1: Quantitative Performance Benchmarks

Test Case	**Nodes Expanded**	**Time (ms)**	**Path Length**
Case 1: Simple (10×10)	38	0.13	13
Case 2: Medium (20×20)	35	0.24	8
Case 3: Dense (30×30)	209	2.55	31

5 Visualization of Results

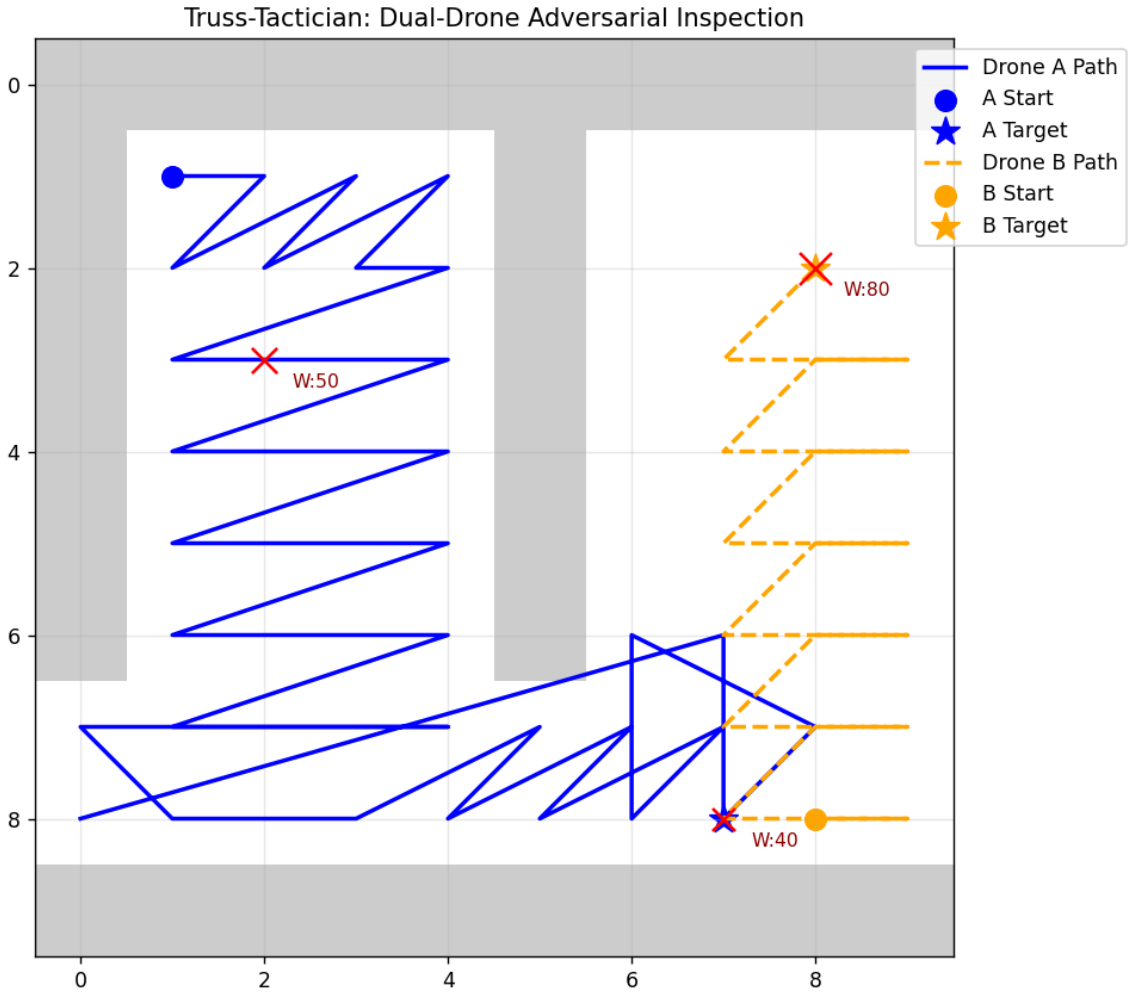

Figure 1: Case 1: 10×10 Pratt Truss

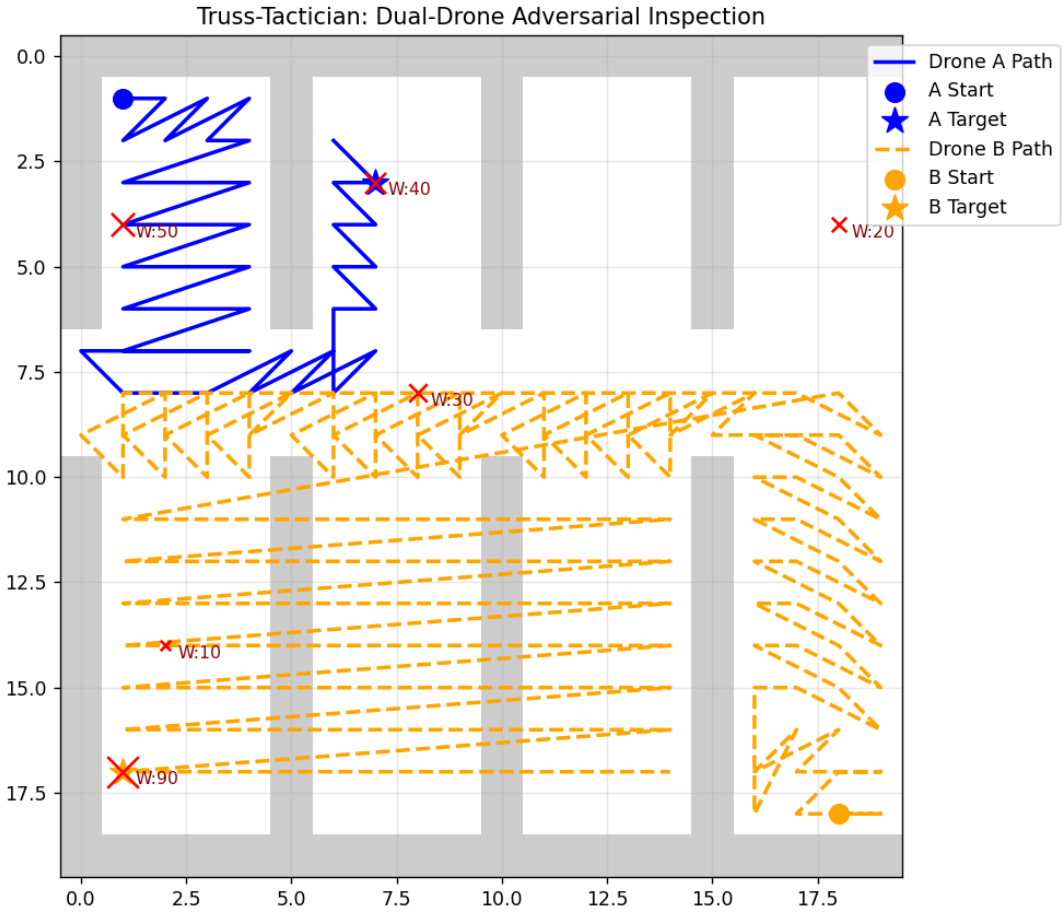

Figure 2: Case 2: 20×20 Pratt Truss

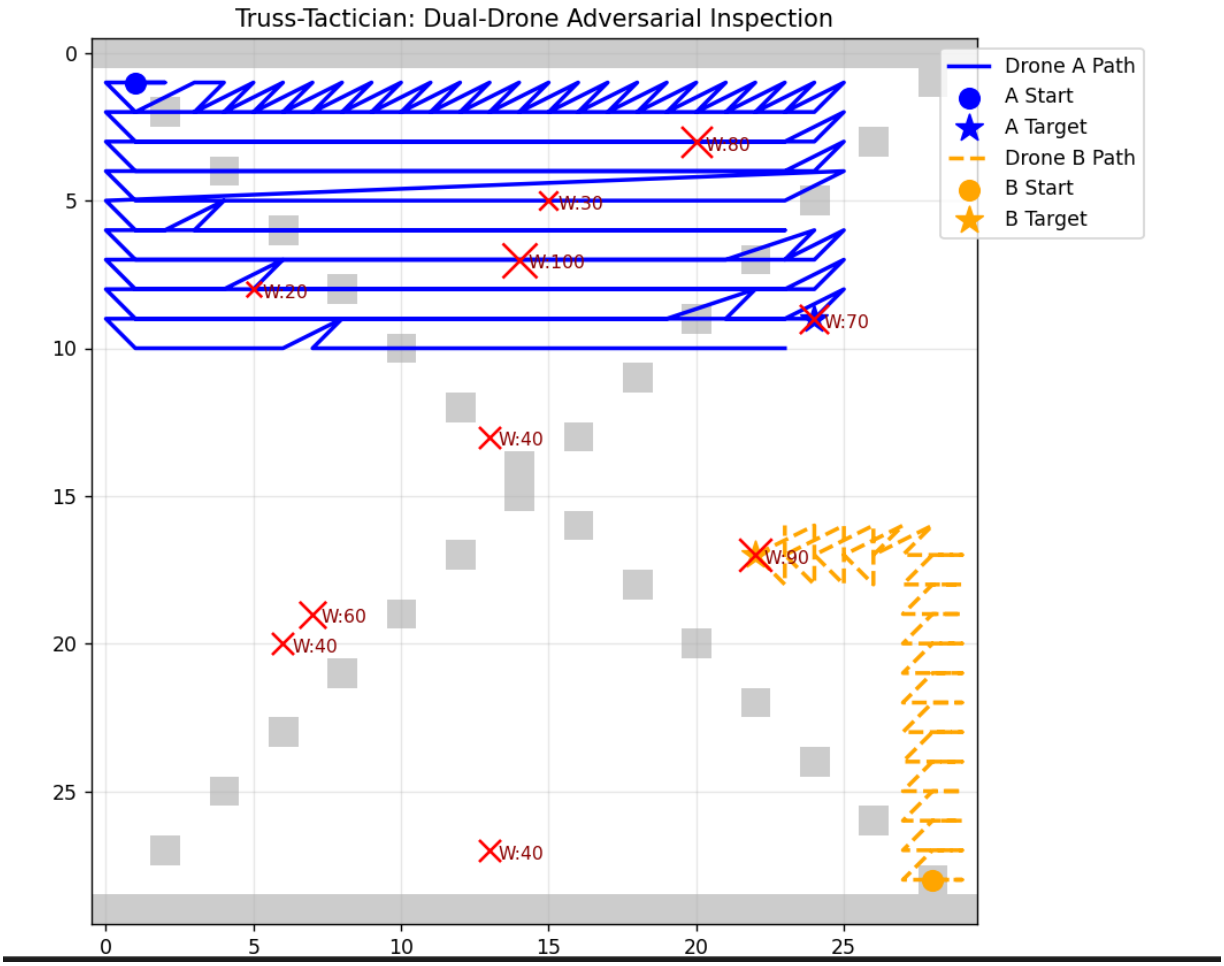

Figure 3: Case 3: 30×30 Warren Truss